CLOUD ENGINEERING ASSIGNMENT

EpicBook

Terraform + Ansible Deployment Guide

A complete, step-by-step walkthrough of provisioning AWS infrastructure with Terraform, configuring a production-style server with Ansible, and deploying a Node.js app with Nginx, PM2 and RDS MySQL.

CLOUD PROVIDER	AWS (EC2 + RDS)
TOOLS	Terraform | Ansible | Nginx | PM2
DATABASE	MySQL on RDS
APPLICATION	EpicBook Node.js

Shared for educational purposes on LinkedIn

Follow along, learn, build, and share with someone who needs it.

11 TASKS
covered end-to-end

Contents

Architecture Overview
Task 1 — Project Folder Layout
Task 2 — Provision Infrastructure with Terraform
Task 3 — Verify SSH Key-Based Access
Task 4 — Ansible Inventory and Configuration
Task 5 — Main Ansible Playbook (site.yml)
Task 6 — The common Role
Task 7 — The nginx Role
Task 8 — The epicbook Role
Task 9 — Group Variables and Ansible Vault
Task 10 — Run the Ansible Playbook
Task 11 — Verify the Deployment
Final Assignment Questions (10)
Next-Time Quick Checklist (29 steps)
The Big Lessons

Architecture Overview

Before diving into the tasks, here is the full picture of what we built and how all the pieces connect.

LAYER	COMPONENT	ROLE
Infrastructure	Terraform	Creates all AWS resources
Compute	EC2 Ubuntu VM	Application server
Web Server	Nginx (port 80)	Reverse proxy / front door
Application	EpicBook Node.js (port 8080)	The actual app
Process Mgr	PM2	Keeps Node.js running
Database	RDS MySQL	Managed relational database
Config / Deploy	Ansible (3 roles)	Configures the EC2 server
Secrets	Ansible Vault	Encrypts sensitive credentials

Request flow when a user visits the app:

```
User Browser
|
[Internet]
|
Nginx :80 (EC2 Public IP)
|
EpicBook Node.js :8080 (local)
|
RDS MySQL (private — NOT public)
```

Note: *RDS is never exposed to the internet. Only the EC2 instance can reach it through private networking. This is intentional and important for security.*

TASK 1 Set Up the Project Folder Layout

```
epicbook-prod/
■■■ terraform/
■■■ ansible/
■■■ site.yml
■■■ inventory.ini
■■■ group_vars/
■ ■■■ web/
■ ■■■ vars.yml
■ ■■■ vault.yml
■■■ roles/
■■■ common/
■■■ nginx/
■■■ epicbook/
```

Q: Which cloud provider did you use?

AWS. An EC2 Ubuntu VM serves as the application server and an RDS MySQL instance handles the managed database.

Q: Why keep Terraform and Ansible in separate folders?

They do different jobs. Terraform creates infrastructure; Ansible configures it. Separation makes the project easier to understand, maintain and troubleshoot.

Q: What is the purpose of the roles directory?

It holds reusable pieces of Ansible configuration. Instead of one large playbook, responsibilities are split into roles: common, nginx and epicbook.

TASK 2 Provision Infrastructure with Terraform

```
terraform init # Download providers
terraform validate # Check syntax
terraform plan # Preview changes — ALWAYS do this first
terraform apply # Create resources
terraform output # Show IPs and endpoints
```

Resource	Purpose
EC2 Ubuntu VM	Application server
RDS MySQL	Managed database (private)
VPC / Subnets	Network isolation
Security Groups	Firewall rules
SSH Key Pair	Secure access to EC2
Terraform Outputs	Expose IP / DB endpoint

Q: Why review terraform plan before apply?

It shows exactly what Terraform intends to create, change or destroy before it touches any real resources. This helps catch mistakes early.

Q: Why should database passwords not appear in Terraform output?

If they show up in screenshots or public repositories, someone could use them to access the database. Mark sensitive outputs with sensitive = true.

Key Lesson: If an EC2 instance is stopped and restarted without an Elastic IP, its public IPv4 address changes. Always check the current IP in AWS before blaming your Ansible config.

TASK 3 Verify SSH Key-Based Access

```
chmod 400 "/path/to/epicbook-key.pem"
ssh -i "/path/to/epicbook-key.pem" ubuntu@<PUBLIC_IP>

# Success looks like:
ubuntu@ip-10-0-1-166:~$
```

Q: What proves SSH key-based access worked?

Connecting without entering a password. The Ubuntu terminal prompt appearing confirms the key was accepted.

Q: What to check if SSH returns Permission denied (publickey)?

1) Correct .pem file 2) chmod 400 on the key 3) Correct username: ubuntu 4) Correct IP 5) Security group allows port 22 6) Key matches the EC2 key pair

TASK 4 Create Ansible Inventory and Configuration

inventory.ini

```
[web]
epicbook ansible_host=54.78.155.244

[web:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=/path/to/epicbook-key.pem
```

Verify before running a playbook:

```
ansible-inventory -i inventory.ini --graph
# @all:
# |--@web:
# | |--epicbook

ansible web -i inventory.ini -m ping
# epicbook | SUCCESS ... pong
```

Q: What is the purpose of inventory.ini?

It tells Ansible which servers to manage and how to connect to them. The EC2 server goes under the web group.

Q: Why is host_key_checking = False used only for this lab?

It prevents Ansible stopping due to SSH host key verification. In production, keep it on — the check is an important security control.

TASK 5 Create the Main Ansible Playbook (site.yml)

```
---
- name: Deploy EpicBook
hosts: web
become: true
roles:
- common
- nginx
```

```
- epicbook
```

Q: What is the purpose of site.yml?

It is the main playbook. It ties the roles together and controls the order they run in.

Q: Why run roles in the order common, nginx, epicbook?

Each stage prepares something needed by the next. common sets up the server, nginx installs the web server, epicbook deploys the app on top of that.

Q: What does become: true do?

Elevates privileges using sudo. Required for installing packages, configuring Nginx and managing system services.

TASK 6 Create the common Role

```
- name: Update apt cache
apt:
update_cache: yes
upgrade: safe

- name: Install common packages
apt:
name:
- git
- curl
- build-essential
- mysql-client
state: present
```

Q: Why should Nginx installation not be in the common role?

Nginx has its own responsibility. common = general server prep, nginx = web server config. Keeping them separate makes the project modular and easier to maintain.

Q: Why is mysql-client useful here?

It allows the EC2 server to connect to and verify the RDS MySQL database from the command line.

TASK 7 Create the nginx Role

Role structure:

```
roles/nginx/
■■■ tasks/
■ ■■■ main.yml
■■■ handlers/
■ ■■■ main.yml # Reload nginx handler goes HERE
■■■ templates/
■■■ epicbook.conf.j2
```

epicbook.conf.j2 (reverse proxy):

```
server {
listen 80;
server_name {{ server_name }};

location / {
proxy_pass http://127.0.0.1:{{ app_port }};
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
}
```

```
}
```

Q: Why is Nginx a reverse proxy here?

It receives user requests on port 80 and forwards them to Node.js on port 8080. Users visit the site normally without knowing the internal port.

Q: Important handler lesson?

The 'Reload nginx' handler MUST go in roles/nginx/handlers/main.yml, NOT inside tasks/main.yml. This is a very common mistake that causes a handler not found error.

TASK 8 Create the epicbook Role

1	Verify Node.js and npm are available
2	Install PM2 globally
3	Create the application directory
4	Clone the application repository from Git
5	Run npm install to get dependencies
6	Create the .env file with DB connection details
7	Start the application with PM2
8	Save the PM2 process list for auto-restart

Q: Why is PM2 used?

PM2 manages the Node.js process. It shows whether the app is running (online), can restart it if it crashes and survives reboots when saved.

Q: Why not hard-code the database password in the role?

Anyone who sees the file gets database access. The password is stored in Ansible Vault and injected at runtime via the .env file.

Q: What does running on :8080 while Nginx listens on :80 mean?

Node.js handles requests internally on 8080. Nginx accepts public traffic on 80 and proxies it. Users visit http:// not http://:8080.

TASK 9 Group Variables and Ansible Vault

group_vars/web/vars.yml

```
app_repo: https://github.com/your-org/epicbook.git
app_dir: /var/www/epicbook
app_port: 8080
pm2_app_name: epicbook
server_name: _
db_host: "{{ rds_endpoint }}"
db_name: epicbook
db_user: epicbook_user
db_password: "{{ vault_db_password }}"
```

group_vars/web/vault.yml (encrypted)

```
# Create: ansible-vault create group_vars/web/vault.yml
# Edit: ansible-vault edit group_vars/web/vault.yml
```

```
# Contents (stored encrypted on disk):
vault_db_password: your-actual-secret-password
```

Note: *vault.yml is encrypted on disk. Never commit the raw password to Git. vars.yml references {{ vault_db_password }} and Ansible merges them at runtime when you supply the vault password.*

TASK 10 Run the Ansible Playbook

1. Syntax check first — always

```
ansible-playbook -i inventory.ini site.yml \
--syntax-check --ask-vault-pass
# Expected: playbook: site.yml
```

2. Run the playbook

```
ansible-playbook -i inventory.ini site.yml --ask-vault-pass
```

3. The success indicator

```
PLAY RECAP
epicbook : ok=XX changed=XX unreachable=0 failed=0
^^^^^^^^
This is what matters
```

Post-playbook verification:

Nginx running?

```
ansible web -i inventory.ini -m command -a "systemctl is-active nginx" --become
--ask-vault-pass

# Expected: active
```

PM2 status?

```
ansible web -i inventory.ini -m command -a "pm2 status" --ask-vault-pass

# Expected: epicbook | online
```

Node.js on 8080?

```
ansible web -i inventory.ini -m command -a "curl -I http://localhost:8080" --ask-vault-pass

# Expected: HTTP/1.1 200 OK
```

TASK 11 Verify the EpicBook Deployment

Main page

```
curl -I http://54.78.155.244

# Expected: HTTP/1.1 200 OK
```

Cart API

```
curl -i --max-time 10 http://54.78.155.244/api/cart

# Expected: HTTP/1.1 200 OK
{"cart":[],"book":null}
```

Cart page

```
curl -o /dev/null -s -w "%{http_code}\n" http://54.78.155.244/cart

# Expected: 200
```

Note: *The cart returning {"cart":[],"book":null} is correct. A 200 OK proves the app and database connection are working.*

Q: What issue was faced during verification?

The EC2 public IP changed when the instance restarted (no Elastic IP). Solution: check AWS for the current IP, update inventory.ini and re-run the playbook.

Final Assignment Questions

All 10 questions with full answers.

1. Why is Terraform used for infrastructure provisioning?

Terraform manages infrastructure as code. Instead of manually creating AWS resources, I define them in files and use terraform plan and terraform apply to build them. This makes infrastructure repeatable, version-controlled and easy to manage.

2. Why are Ansible roles useful for production-style deployments?

Roles break the deployment into smaller, reusable sections. Each role is focused: common (server prep), nginx (web server), epicbook (app). Each can be maintained or reused independently.

3. What is the purpose of group_vars/web.yml?

It stores shared variables for the web group. Values like app_port, app_repo, db_host and pm2_app_name are defined once and reused by all roles.

4. Why should database passwords not be committed to GitHub?

They are sensitive credentials. If committed to a public repo, anyone could use them. Ansible Vault encrypts secrets at rest so they can be safely stored.

5. What is the purpose of Nginx in this deployment?

Nginx is a reverse proxy. It accepts public traffic on port 80 and forwards it to Node.js on port 8080. This hides the internal port from users.

6. Why should the managed MySQL database not be publicly accessible?

The database should only be reachable by the application server. Keeping it private reduces the attack surface and prevents direct internet access to MySQL.

7. Why is PM2 used for the Node.js application?

PM2 manages the Node.js process. It shows runtime status (online), can restart on crash and survives reboots when configured with pm2 save and pm2 startup.

8. What does idempotency mean in Ansible?

Running the same playbook multiple times should not change things that are already correct. If Nginx is already installed, Ansible skips it. Ansible modules are idempotent by design — this is why they are preferred over raw shell commands.

9. What issues were faced and how were they fixed?

Issues: EC2 IP changed, Ansible Vault misconfiguration, missing community.general collection, Nginx handler not found, Node/npm package issues. Fixes: updated the IP in inventory.ini, corrected Vault setup, installed the collection, moved the handler to handlers/main.yml and used the pre-existing Node.js installation. Reran until failed=0.

10. What security improvements before going to production?

Restrict SSH to trusted IPs only | Keep RDS private (already done) | Use SSL/HTTPS with a real domain | Use a proper secrets manager | Apply least-privilege IAM | Rotate credentials regularly | Audit and tighten security group rules | Avoid sensitive data in logs

Next-Time Quick Checklist

Follow this exact order on your next infrastructure assignment.

1	Create the project folder structure
2	Write Terraform files (EC2, RDS, networking, security groups)
3	terraform init
4	terraform validate
5	terraform plan — review carefully before proceeding
6	terraform apply
7	terraform output — note the EC2 IP and DB endpoint
8	Find the current EC2 public IP from AWS console
9	chmod 400 the .pem key file
10	Test SSH: ssh -i key.pem ubuntu@<IP>
11	Activate your Ansible virtual environment (.venv)
12	Update inventory.ini with the current EC2 IP
13	ansible-inventory -i inventory.ini --graph
14	ansible web -i inventory.ini -m ping
15	Create site.yml
16	Create the common role
17	Create the nginx role (handler in handlers/main.yml)
18	Create the epicbook role
19	Create group_vars/web/vars.yml
20	Create and encrypt group_vars/web/vault.yml with the DB password
21	ansible-playbook --syntax-check --ask-vault-pass
22	ansible-playbook -i inventory.ini site.yml --ask-vault-pass
23	Confirm PLAY RECAP shows failed=0
24	Verify Nginx is active
25	Verify PM2 shows epicbook as online
26	Verify Node.js responds on port 8080
27	curl -I http://<PUBLIC_IP> — check for 200 OK
28	curl the /api/cart endpoint — check for 200 OK
29	Open the app in the browser

If something breaks, check these first:

!	CHECK	HOW
1	**Did the EC2 public IP change?**	Check AWS console, update inventory.ini
2	**Is SSH still working?**	ssh -i key.pem ubuntu@
3	**Is PM2 / the app running?**	pm2 status
4	**Is Nginx active?**	systemctl is-active nginx
5	**Is the Ansible venv activated?**	source .venv/bin/activate

The Big Lessons from This Project

If you forget everything else, keep these.

Terraform = Infrastructure	Creates EC2, RDS, networking, security groups. Nothing else.
Ansible = Configuration	Configures the server Terraform created. Never creates AWS resources.
Roles = Organisation	common prepares server. nginx sets up web server. epicbook deploys app.
Nginx = Front Door	Traffic hits Nginx on :80. Nginx proxies it to Node.js on :8080.
PM2 = Process Manager	Keeps Node.js alive, shows status, and survives reboots.
RDS = Database	Managed MySQL. Private. Never exposed to the internet.
Ansible Vault = Secrets	Encrypts passwords at rest. Reference with {{ vault_db_password }}.
Inventory = Connection Map	Tells Ansible: where is the server, which user, which SSH key.
failed=0 = Success	The only Ansible PLAY RECAP line that matters.
Elastic IP = Stability	Without one, EC2 public IP changes on every restart. Use it in production.

This guide was put together from a real assignment. Every issue, fix and lesson here came from actually doing the work. If you find it useful, share it, build on it, and pass it on to someone who needs it.